IEnumerable, IQueryable, Lists

Which? When? Why? What is the cost when you made a mistake?

We've all made a mistake here at least once and used `.ToList()` everywhere. Here's when to use which interface, why it matters, and how much it costs you if you get it wrong.

Swipe

Let's get started

Introduction to these Interfaces & Classes

IEnumerable

The most basic interface for iterating over a collection in memory using a forward-only cursor.

IQueryable

Designed for out-of-memory data sources (like databases), allowing LINQ queries to be converted into optimized SQL.

List

A concrete class that holds data in memory, providing fast index-based access and immediate execution.

Simplifed version

Can we do an... Apple analogy?

IEnumerable(Tree)

You see the apples, but you must pick them one by one to inspect them. Filtering happens at your doorstep (In-Memory).

IQueryable(Order Sheet)

You send a specific request to the farmer: "I only want 5 red apples." The farmer (Database) does the work and sends only what you asked for.

List(The Crate)

The apples are already picked, counted, and sitting in your kitchen. Ready to use, but they take up space (RAM).

Under the hood

Explanation via code (EF only)

```
// IEnumerable – iteration in memory
IEnumerable<string> names = new List<string> { "Ana",
"Edo", "Mia" };
var filtered = names.Where(n ⇒ n.StartsWith("A"));
// filter in C# memory
```

```
// IQueryable – expression tree, building SQL
IQueryable<User> query = dbContext.Users;
var filter = query.Where(u ⇒ u.Name.StartsWith("A"));
// SQL: SELECT * FROM users WHERE name LIKE 'A%'
```

```
// List – everything in memory at the moment
List<string> list = names.ToList();
// materialization
```

Deferred Execution

Queries are execcuted when they are realized

This is the most important concept. Neither IEnumerable nor IQueryable do anything until you call `ToList()`, `foreach`, `Count()`, etc.

```
var query = dbContext.Songs.Where(s ⇒ s.Artist ==
"Queen");
// Nothing happens. No SQL query.

query = query.OrderBy(s ⇒ s.Title);
// Still nothing.

var result = query.ToList();
// NOW goes SQL query. With all filters at once.
```

N+1 Problem

The most expsensive mistake

```
// WRONG — IEnumerable u loop
IEnumerable<Artist> artists = repo.GetAllArtists();
// 1 query

foreach (var artist in artists)
{
    var songs = repo.GetSongsByArtist(artist.Id);
    // N queries
    // If we have 100 artists = 101 queries for DB
}

// CORRECT — one JOIN query
var artistsWithSongs = repo.GetAllArtistsWithSongs();
// 1 query
```

Conclusion

Desicion table

Situation	Use
Repo method returns data	IEnumerable<T>
Build query step-by-step (EF core)	IQueryable<T>
Read once, move on	IEnumerable<T>
Needs Count, [], operator...	List<T>
API Response	IEnumerable<T> List<T>
Dapper result	IEnumerable<T>